The Dispatch Substrate Kernel: Architecture, Composition, and Conformance of a Five-Subsystem Dispatch Loop for Categorical Computation

Kundai Farai Sachikonye
Technical University of Munich
Chair of Proteomics and Bioanalytics
AIMe Registry for Artificial Intelligence
Maximus-von-Imhof-Forum 3, 85354 Freising, Germany
kundai.sachikonye@wzw.tum.de

June 4, 2026

Abstract

We specify a kernel architecture for the dispatch of categorical computation: an execution substrate that takes a symbolic fragment representing an operation and returns the value the operation produces. The substrate wraps an executor with five subsystems that derive from the canonical decomposition of any dispatch responsibility: pre-dispatch validation, post-dispatch memoization, pending-work scheduling, external-resource retrieval, and runtime invariant monitoring. We formalise each subsystem as an interface with a small, frozen surface, define the dispatch loop that composes them with the underlying executor, and prove that the composition is sound and deterministic. We further establish four structural results. (i) The *Dispatch Correctness Theorem*: with all five subsystems instantiated to identity (no-op), the kernel produces results bit-identical to the bare executor; with non-trivial subsystems, the result is preserved modulo the subsystems' specified semantics. (ii) The *Independence Theorem*: the five subsystems are pairwise functionally independent, in the sense that the dispatch result depends on each subsystem only through its specified interface and not through shared internal state. (iii) The *Stability Contract Theorem*: extending the kernel with additional subsystems (or strengthening existing ones) does not invalidate any client that depends only on the frozen interface surface. (iv) The *Conformance Decidability Theorem*: the property-based regression suite over kernel observable events is decidable, and the kernel admits cross-architecture invariance under a fixed conformance methodology. We further give the dispatch algorithm explicitly, analyse its time complexity ($\mathcal{O}(1)$ per subsystem hook plus the executor's intrinsic cost), and characterise the parallelism opportunities admitted by the architecture. The kernel is purely a substrate: it does not commit to a particular symbolic language, a particular cache eviction policy, a particular scheduler, or a particular retrieval mechanism. What it provides is the architectural skeleton over which such choices are made, together with the theorems that ensure the choices do not invalidate the substrate's guarantees.

Keywords: dispatch substrate, kernel architecture, microkernel, validation engine, memoization, scheduling, surgical retrieval, conservation monitoring, stability contract, conformance regression.

Contents

Part I
Preliminaries

1 Introduction

1.1 The dispatch problem

A dispatch substrate is the runtime component that, given a symbolic representation of a computation, produces its value. The category of substrates is broad: an operating-system kernel dispatches a system call to its handler (Tanenbaum and Bos, 2014; Liedtke, 1995; Klein et al., 2009); a virtual machine dispatches a bytecode instruction to its interpreter (Lindholm et al., 2014); an actor system dispatches a message to a registered receiver (Hewitt et al., 1973; Agha, 1986; Armstrong, 2003); a microservice gateway dispatches an RPC to a service implementation (Birrell and Nelson, 1984); a database engine dispatches a query plan to a physical operator (Graefe, 1993). Each of these substrates handles dispatch under different physical assumptions but shares a common architectural decomposition.

We make this decomposition precise and present it as a single design: a kernel that wraps an executor with exactly five subsystems whose responsibilities partition the dispatch envelope. The five are derived from the universal dispatch responsibilities and not from any particular substrate; together they cover the architectural surface of any dispatch component, and individually they admit independent substitution and refinement.

1.2 The five responsibilities

Every dispatch substrate has the following five responsibilities, regardless of its application domain.

Pre-dispatch validation. Before evaluation begins, the substrate must determine whether the input fragment is admissible. This responsibility separates well-formed inputs from inputs that would cause the executor to misbehave. We name the subsystem implementing it the *proof-validation engine* (PVE).

Post-dispatch memoization. After evaluation completes, the substrate may cache the result against a fingerprint of the input to avoid re-evaluating the same fragment in the future. This responsibility realises the cache-hit speedup that distinguishes a memoising dispatch substrate from a stateless one. We name the subsystem implementing it the *categorical memory manager* (CMM).

Pending-work scheduling. Between pre-dispatch and execution, the substrate may have a backlog of pending fragments to evaluate; it must decide their order. This responsibility is the substrate's scheduler. We name the subsystem implementing it the *pending-state scheduler* (PSS).

External-resource retrieval. During evaluation, the executor may require external data—bytes from a network, columns from a database, weights from a model. The substrate mediates this retrieval and may impose a retrieval policy that fetches less than the executor would naively request. We name the subsystem implementing it the *demon I/O controller* (DIC).

Runtime invariant monitoring. After dispatch completes, the substrate may publish observable events that downstream observers consume to check invariants (monotone counters, conservation laws, regression-test predicates). The monitoring is asynchronous and lossy: a slow observer does not throttle the dispatch path. We name the subsystem implementing it the *trigger event monitor* (TEM).

We claim that these five are the canonical decomposition. Any dispatch substrate either implements all five (with non-trivial choices of policy) or implements a strict subset (with trivial identity policies for the remaining responsibilities). No substrate requires a sixth responsibility that is not expressible as a composition of these five.

1.3 Overview of main results

The paper establishes the following:

1. **Theorem 5.2** (Dispatch Correctness): the dispatch loop with all subsystems instantiated to identity returns bit-identical results to the bare executor.
2. **Theorem 6.2** (Subsystem Independence): the five subsystems are pairwise functionally independent through their interfaces.
3. **Theorem 7.3** (Stability Contract): clients depending only on the frozen interface surface remain functional under any additive evolution of the kernel.
4. **Theorem ??** (Determinism): if each subsystem is a deterministic function of its inputs, so is the kernel's dispatch output.
5. **Theorem 8.1** (Compositional Substitution): swapping a subsystem for a bisimilar one does not change the dispatch result.
6. **Theorem 9.3** (Conformance Decidability): the property-based regression suite over kernel observables is decidable in time linear in the workload size.

2 The Underlying Executor

The kernel wraps an executor. We treat the executor as a black box characterised by its interface; the kernel's properties will be proved relative to that interface.

Definition 2.1 (Executor)**.** *An* executor *is a triple* $E = \langle \mathcal{F}, \mathcal{V}, \mathrm{eval} \rangle$ *where* $\mathcal{F}$ *is a set of* fragments *(symbolic representations of computations),* $\mathcal{V}$ *is a set of* values*, and* $\mathrm{eval} : \mathcal{F} \to \mathcal{V} \cup \{\bot\}$ *is the* evaluation function*. The symbol* $\bot$ *denotes evaluation failure (e.g. unresolved hole, missing provider).*

Remark 2.2. *We deliberately leave the structure of* $\mathcal{F}$ *and* $\mathcal{V}$ *abstract. The kernel architecture is parametric in the executor: the same five subsystems apply to a fragment of an arithmetic-expression language, a process-calculus term, or an RPC payload. Specific fragment grammars and value spaces are application-specific and need not be fixed at the architectural level.*

Definition 2.3 (Determinism)**.** *The executor is* deterministic *if* eval *is a function (single-valued); i.e. for every* $F \in \mathcal{F}$*,* $\mathrm{eval}(F)$ *takes a single value (possibly* $\bot$*).*

Definition 2.4 (Operation Registry)**.** *An* operation registry $\mathcal{R}$ *is a finite map from named operation identifiers to* providers*: implementations that accept named-argument records of values and return values. The registry mediates how* eval *resolves named-operation references within a fragment.*

We assume E has a registry $\mathcal{R}$ as an implicit parameter and that eval uses $\mathcal{R}$ to dispatch named-operation calls.

Part II
The Five Subsystems

3 Subsystem Interfaces

We now define each of the five subsystems by its interface. Each definition is parametric in the executor type and contains the minimal set of methods the dispatch loop requires.

3.1 PVE: Pre-Dispatch Validation

Definition 3.1 (Validation Subsystem)**.** *A* validation subsystem σ_{P} *is an object exposing a method*

$$\mathrm{valid} : \mathcal{F} \to \mathbb{B},$$

where $\mathrm{valid}(F) = \mathrm{true}$ *indicates that the fragment is admissible for dispatch and* false *indicates that the dispatch path must reject the fragment without evaluation.*

Definition 3.2 (Identity PVE)**.** *The* identity validation subsystem $\sigma_{\mathrm{P},0}$ *returns* true *for every fragment: it imposes no validation constraint.*

The validation subsystem is the substrate's gatekeeper. Concrete instances may implement type checking, refinement-type discipline, three-route verification, capability checks, signature validation, or any other admission criterion expressible as a Boolean fragment predicate. The interface is opaque to the specific check.

3.2 CMM: Post-Dispatch Memoization

Definition 3.3 (Memoization Subsystem)**.** *A* memoization subsystem σ_{M} *is an object exposing two methods:*

$$\mathrm{lookup} : \mathcal{F} \to \mathcal{V} \cup \{\mathrm{miss}\},$$
$$\mathit{insert} : \mathcal{F} \times \mathcal{V} \to \mathbf{1}.$$

The lookup *method returns a previously cached value for the given fragment or signals a miss; the insert method stores a fresh result. The result of insert is unit (no semantic value); the side-effect is internal.*

Definition 3.4 (Identity CMM)**.** *The* identity memoization subsystem $\sigma_{\mathrm{M},0}$ *always returns* miss *from* lookup *and is a no-op on insert.*

3.3 PSS: Pending-State Scheduling

Definition 3.5 (Scheduling Subsystem)**.** *A* scheduling subsystem σ_{S} *is an object exposing a method*

$$\mathit{order} : \mathcal{F}^* \to \mathcal{F}^*,$$

where $\mathcal{F}^*$ *is the set of finite sequences of fragments, and the output is a permutation of the input that specifies the order in which the pending fragments should be evaluated.*

Definition 3.6 (Identity PSS)**.** *The* identity scheduling subsystem $\sigma_{\mathrm{S},0}$ *returns the input sequence unchanged: it preserves the arrival order of pending fragments.*

3.4 DIC: External-Resource Retrieval

Definition 3.7 (Retrieval Subsystem)**.** *A* retrieval subsystem σ_{R} *is an object exposing a method*

$$\mathit{policy} : Q \times S \to P,$$

where Q *is the set of queries the executor may issue,* S *is the set of external sources, and* P *is the set of retrieval policies (e.g. "fetch all", "fetch the top-k mutual-information records"). The executor consults* σ_{R} *before issuing an external request and follows the returned policy.*

Definition 3.8 (Identity DIC)**.** *The* identity retrieval subsystem $\sigma_{\mathrm{R},0}$ *always returns the "fetch all" policy: the executor retrieves whatever it would have retrieved without surgical mediation.*

3.5 TEM: Runtime Invariant Monitoring

Definition 3.9 (Monitoring Subsystem)**.** *A* monitoring subsystem σ_{T} *is an object exposing a method*

$$\mathit{observe} : \mathcal{E} \to \mathbf{1},$$

where $\mathcal{E}$ *is the set of* dispatch events*: structured records published by the dispatch loop on each*

completed dispatch. The result is unit; the side-effect is the monitor's internal book-keeping (typically asynchronous, non-blocking, lossy under back-pressure).

Definition 3.10 (Identity TEM)**.** *The* identity monitoring subsystem $\sigma_{\mathrm{T},0}$ *is a no-op on observe: it ignores every event.*

4 Dispatch Events

Definition 4.1 (Dispatch Event)**.** *A* dispatch event $e \in \mathcal{E}$ *is a structured record carrying at least the following fields:*

- *the top-level operation name (or $\perp$ for non-Call roots);*
- *the dispatch elapsed time;*
- *the success/failure flag;*
- *the cache-hit flag (whether the result was served from σ_{M});*
- *the decision counter increment (an integer-valued contribution to a monotone counter the kernel maintains).*

The event is the unit of observable kernel activity; the TEM subscribes to a broadcast channel of events and consumes them out-of-band.

Part III
The Dispatch Loop

5 The Dispatch Algorithm

5.1 Composition

Definition 5.1 (Kernel)**.** *A* kernel $\mathcal{K}$ *is a tuple*

$$\mathcal{K} = \langle E, \sigma_{\mathrm{P}}, \sigma_{\mathrm{M}}, \sigma_{\mathrm{S}}, \sigma_{\mathrm{R}}, \sigma_{\mathrm{T}}, \mathcal{E} \rangle$$

consisting of an executor E, the four subsystems of Definitions 3.1–3.7, the monitor σ_{T}, and a dispatch-event channel $\mathcal{E}$.

5.2 The dispatch procedure

Algorithm 1 Kernel Dispatch

```
1:  function DISPATCH(K, F)
2:      t0 ← clock now
3:      if not σP.valid(F) then
4:          publish E(op = ⊤(F), ok = false, cache = false)
5:          return ⊥
6:      end if
7:      ordered ← σS.order(σS-queue ∪ {F})
8:      F ← head(ordered)
9:      r ← σM.lookup(F)
10:     if r ≠ miss then
11:         v ← r;  cache-hit ← true
12:     else
13:         v ← E.eval(F)  ▷ may consult σR for I/O
14:         σM.insert(F, v)
15:         cache-hit ← false
16:     end if
17:     t1 ← clock now
18:     e ← E(op = ⊤(F), elapsed = t1 − t0,
19:           ok = v ≠ ⊥, cache = cache-hit)
20:     publish e
21:     σT.observe(e)
22:     return v
23: end function
```

5.3 Soundness

Theorem 5.2 (Dispatch Correctness with Identity Subsystems)**.** *Let $\mathcal{K}_0 = \langle E, \sigma_{\mathrm{P},0}, \sigma_{\mathrm{M},0}, \sigma_{\mathrm{S},0}, \sigma_{\mathrm{R},0}, \sigma_{\mathrm{T},0}, \mathcal{E} \rangle$ be a kernel with all subsystems instantiated to identity. Then for every fragment F:*

$$\mathrm{dispatch}(\mathcal{K}_0, F) = \mathrm{eval}(F).$$

Proof. We trace Algorithm 1 with the identity subsystems substituted.

Line 3: $\sigma_{\mathrm{P},0}.\mathrm{valid}(F) = \mathrm{true}$ unconditionally; the early return is not taken.

Line 7: $\sigma_{\mathrm{S},0}$.order returns the input unchanged; the head is F.

Line 8: $\sigma_{\mathrm{M},0}.\mathrm{lookup}(F) = \mathrm{miss}$ unconditionally; the cache-hit branch is not taken.

Line 12: the executor is invoked: $v \leftarrow \mathrm{eval}(F)$.

Line 13: $\sigma_{\mathrm{M},0}$.insert is a no-op.

Lines 16–19: $\sigma_{\mathrm{T},0}$.observe is a no-op; the event is published on the channel but has no semantic effect on the return value.

Line 20: return $v = \text{eval}(F)$.
Hence $\text{dispatch}(\mathcal{K}_0, F) = \text{eval}(F)$. □ □

Theorem 5.3 (Dispatch Correctness with Non-Trivial Subsystems)**.** *Let $\mathcal{K} = \langle E, \sigma_\text{P}, \sigma_\text{M}, \sigma_\text{S}, \sigma_\text{R}, \sigma_\text{T}, \mathcal{E} \rangle$ be a kernel with arbitrary subsystems. Then for every fragment F admitted by σ_P for which σ_M contains no prior entry,* $\text{dispatch}(\mathcal{K}, F) = \text{eval}(F)$*. For an admitted fragment for which σ_M contains a prior entry v_0 from a previous dispatch of F,* $\text{dispatch}(\mathcal{K}, F) = v_0$.

Proof. Cache miss: Lines 7–12 proceed as in Theorem 5.2, yielding $v = \text{eval}(F)$.

Cache hit: Line 9 returns the cached value $r = v_0$, and line 19 returns $v = v_0$.

In both cases the rejection branch (line 3) does not fire because $\sigma_\text{P}.\,\text{valid}(F) = \text{true}$ by hypothesis. □ □

Corollary 5.4 (Determinism Preservation)**.** *If E is deterministic and each subsystem is a deterministic function of its inputs, then* $\text{dispatch}(\mathcal{K}, \cdot)$ *is a deterministic function on $\mathcal{F}$.*

Proof. By Theorem 5.3, the dispatch result is determined by $(F, \sigma_\text{P}.\,\text{valid}(F), \sigma_\text{M}.\,\text{lookup}(F), \text{eval}(F))$. Each component is a deterministic function of its inputs by hypothesis. The composition is therefore deterministic. □ □

Part IV
Structural Properties

6 Subsystem Independence

6.1 Functional independence

Definition 6.1 (Functional Independence)**.** *Two subsystems σ_a, σ_b are* functionally independent *in a kernel $\mathcal{K}$ if the dispatch output* $\text{dispatch}(\mathcal{K}, F)$ *depends on σ_a only through the values returned by σ_a's interface methods, and similarly for σ_b. In particular, swapping σ_a for any σ'_a that returns identical interface outputs on the dispatched fragment does not change the dispatch result.*

Theorem 6.2 (Pairwise Independence)**.** *The five subsystems $\sigma_\text{P}, \sigma_\text{M}, \sigma_\text{S}, \sigma_\text{R}, \sigma_\text{T}$ in Algorithm 1 are pairwise functionally independent.*

Proof. We inspect each pair.

$(\sigma_\text{P}, \sigma_\text{M})$*:* σ_P is consulted at line 3 and its return value b alone determines whether the rejection branch fires. σ_M is consulted at line 8 only if validation passed (i.e. $b = \text{true}$). The two consultations are sequential and depend on disjoint internal states.

$(\sigma_\text{P}, \sigma_\text{S})$*:* σ_S at line 7 reorders pending fragments; the chosen head is then validated (if it has not been) or proceeds to line 8. Whether validation passes depends on the fragment, not on the scheduling order; the two are independent in the dispatch-result sense.

$(\sigma_\text{P}, \sigma_\text{R})$*:* σ_R is consulted by the executor at line 12; σ_P is consulted at line 3. Their roles do not intersect in the dispatch path.

$(\sigma_\text{P}, \sigma_\text{T})$*:* σ_T consumes events at line 18; its interface is observe-only. The dispatch result is fixed by line 19 before σ_T runs.

$(\sigma_\text{M}, \sigma_\text{S})$*:* σ_S reorders the pending queue; σ_M services per-fragment cache requests. Reordering does not change which fragments are cached.

$(\sigma_\text{M}, \sigma_\text{R})$*:* on a cache hit, the executor is not invoked; σ_R is therefore not consulted. On a cache miss, σ_R is consulted by the executor independently of any prior σ_M state.

$(\sigma_\text{M}, \sigma_\text{T})$*:* σ_T observes events that include the cache-hit flag, but σ_T's observe method does not feed back into σ_M.

$(\sigma_\text{S}, \sigma_\text{R})$*:* σ_S orders pending fragments; σ_R controls per-fragment external retrieval. The orderings and policies are on disjoint axes.

$(\sigma_\text{S}, \sigma_\text{T})$*:* σ_T observes events emitted at the per-dispatch granularity; σ_S chooses the next fragment. The two operate at different layers.

$(\sigma_\text{R}, \sigma_\text{T})$*:* σ_R's policies affect what the executor retrieves; σ_T's observe is post-dispatch and orthogonal.

In every pair, the two subsystems consult their interfaces sequentially or are causally disjoint in the dispatch path. Hence the pair is functionally independent. □ □

Corollary 6.3 (Substitution Theorem)**.** *For any pair of bisimilar subsystems $\sigma_X \sim \sigma'_X$ at any of the five positions, replacing σ_X with σ'_X in $\mathcal{K}$ does not change the dispatch output:*

$$\text{dispatch}(\mathcal{K}[\sigma_X], F) = \text{dispatch}(\mathcal{K}[\sigma'_X], F) \quad \forall F \in \mathcal{F}.$$

Proof. By Definition 6.1, the dispatch output depends on each subsystem through its interface

only. Bisimilarity implies identical interface behaviour. The substitution is therefore semantics-preserving. □ □

7 Stability Contract

7.1 The frozen surface

Definition 7.1 (Frozen Surface)**.** *The* frozen surface *of a kernel $\mathcal{K}$ is the set of method signatures of the five subsystems and the executor's* eval *function. The frozen surface is the contract on which downstream clients depend.*

Definition 7.2 (Evolution)**.** *The kernel* evolves additively *if the frozen surface of an evolved kernel $\mathcal{K}'$ is a superset of the frozen surface of $\mathcal{K}$, with every original method having the same signature in $\mathcal{K}'$ as in $\mathcal{K}$.*

Theorem 7.3 (Stability Contract)**.** *Let $\mathcal{K}'$ be an additive evolution of $\mathcal{K}$ as in Definition 7.2. Then every client whose behaviour depends only on the frozen surface of $\mathcal{K}$ behaves identically when wired to $\mathcal{K}'$ (modulo any newly added optional capabilities the client elects not to use).*

Proof. The client invokes only methods in the frozen surface of $\mathcal{K}$. By the additive-evolution hypothesis, every such method has the same signature in $\mathcal{K}'$. The client's invocations resolve identically; the responses, by the executor and subsystem contracts, are identical (modulo subsystem-specific semantic refinement that the client does not observe). Hence the client's observable behaviour is unchanged. □ □

Remark 7.4. *Theorem 7.3 is the structural foundation of the kernel's release-engineering discipline. Once a method signature has been promoted to the frozen surface, downstream clients have a contract guarantee that the kernel will not break them. The evolution policy is conservative by design: every addition is opt-in, every removal is a major-version event, and every signature change is a breaking change.*

7.2 Refinement of subsystems

Proposition 7.5 (Subsystem Refinement)**.** *A subsystem σ may be replaced by a refined σ' that exposes the same interface and produces interface-equivalent behaviour without breaking any client of the kernel.*

Proof. By Theorem 6.2, the dispatch output depends on σ only through its interface. By Corollary 6.3, interface-bisimilar substitution preserves the dispatch output. Hence the refinement is transparent to all clients of $\mathcal{K}$. □ □

8 Compositionality

Theorem 8.1 (Compositional Substitution)**.** *Let $\mathcal{K}_a, \mathcal{K}_b$ be two kernels with bisimilar subsystems at each of the five positions ($\sigma_{\mathrm{P},a} \sim \sigma_{\mathrm{P},b}$, $\sigma_{\mathrm{M},a} \sim \sigma_{\mathrm{M},b}$, …, $\sigma_{\mathrm{T},a} \sim \sigma_{\mathrm{T},b}$) and identical executors. Then for every fragment $F \in \mathcal{F}$:*

$$\mathrm{dispatch}(\mathcal{K}_a, F) = \mathrm{dispatch}(\mathcal{K}_b, F).$$

Proof. By Corollary 6.3 applied five times in succession (one per subsystem position), the dispatch output is unchanged at each substitution. The composite substitution from $\mathcal{K}_a$ to $\mathcal{K}_b$ is therefore output-preserving. □ □

Corollary 8.2 (Reference-Implementation Equivalence)**.** *Any two implementations of the kernel that respect the five interface contracts and use the same executor produce identical outputs on every fragment.*

Proof. Direct from Theorem 8.1: interface-contract compliance is interface bisimilarity. □ □

Part V
Conformance Regression

9 The Conformance Suite

9.1 Property-based regression

Definition 9.1 (Property-Based Test)**.** *A* property-based test *over a kernel $\mathcal{K}$ is a triple $\langle W, P, \tau \rangle$ where W is a workload (a finite sequence of fragments), P is a Boolean predicate over dispatch-event streams, and τ is an acceptance threshold (real number in $[0, 1]$ indicating the fraction of W on which P must hold).*

Definition 9.2 (Conformance Run)**.** *A* conformance run *of test $\langle W, P, \tau \rangle$ on $\mathcal{K}$ is the procedure that dispatches each fragment in W via $\mathcal{K}$, collects*

the emitted event stream, and reports the fraction of events for which P holds. The run passes *iff this fraction is at least τ.*

Theorem 9.3 (Conformance Decidability)**.** *For a finite workload W and a decidable predicate P, the conformance run of $\langle W, P, \tau\rangle$ on $\mathcal{K}$ is decidable in time $\mathcal{O}(|W| \cdot (T_{\text{dispatch}} + T_P))$, where T_{dispatch} is the per-dispatch cost and T_P is the per-event predicate evaluation cost.*

Proof. Iterate W, dispatch each fragment, evaluate P on the emitted event, count the fraction, compare to τ. Each step has bounded cost; the total cost is the product of $|W|$ and the per-step cost. Termination is guaranteed by W being finite. □ □

9.2 Cross-architecture invariance

Definition 9.4 (Cross-Architecture Invariance)**.** *A conformance test passes* cross-architecture-invariantly *if it passes on every supported host configuration (different CPUs, operating systems, toolchain versions).*

Proposition 9.5 (Floating-Point Bound on Cross-Architecture Discrepancy)**.** *For a conformance test whose predicate P is continuous in numerical fields of the event, the cross-architecture discrepancy in pass-rate is bounded by ϵ_{mach} times the predicate's Lipschitz constant times $|W|$, where ϵ_{mach} is the maximum floating-point machine epsilon across supported hosts.*

Proof. The dispatch output and event-stream fields are computed with floating-point arithmetic; their per-step error is bounded by ϵ_{mach}. The predicate's Lipschitz continuity bounds the resulting variation in P's value. Summing over the workload gives the stated bound. □ □

Remark 9.6. *Proposition 9.5 establishes that cross-architecture invariance is achievable for floating-point-continuous predicates with a precision tolerance scaling linearly in the workload. For integer-valued or Boolean predicates, the discrepancy is zero and cross-architecture pass-rate is exact.*

Part VI
Performance and Parallelism

10 Time Complexity

Theorem 10.1 (Per-Dispatch Time Bound)**.** *The per-dispatch cost of Algorithm 1 is*

$$T_{\text{dispatch}} = T_{\sigma_{\text{P}}} + T_{\sigma_{\text{S}}} + T^{\text{lookup}}_{\sigma_{\text{M}}} + T_E + T^{\text{insert}}_{\sigma_{\text{M}}} + T_{\sigma_{\text{T}}} + \mathcal{O}(1),$$

where T_X is the per-call cost of subsystem X or the executor's intrinsic cost.

Proof. Direct accounting of the algorithm's lines: the five subsystem hooks plus the executor invocation, plus constant-time arithmetic (clock reads, event construction, branch decisions). □ □

Corollary 10.2 (Identity-Subsystem Overhead)**.** *With all subsystems instantiated to identity, the per-dispatch overhead is $\mathcal{O}(1)$ over the bare executor cost T_E.*

Proof. Each identity subsystem has $\mathcal{O}(1)$ method cost: identity Boolean return, identity sequence return, miss return, fetch-all return, no-op observe. Summing five $\mathcal{O}(1)$ contributions plus arithmetic gives the bound. □ □

10.1 Asynchronous monitor

The TEM subsystem is invoked synchronously in Algorithm 1 (line 18), but it is permitted to be a non-blocking event publisher. We make this precise.

Definition 10.3 (Non-Blocking TEM)**.** *A monitoring subsystem σ_{T} is* non-blocking *if its observe method returns in bounded time independent of the rate at which events are published. Implementations include lock-free broadcast channels, ring buffers with overwrite, and bounded-buffer drop-on-overflow.*

Proposition 10.4 (Back-Pressure Isolation)**.** *A non-blocking σ_{T} ensures that slow observers do not throttle the dispatch path. The per-dispatch cost of observe is independent of the observer's processing rate.*

Proof. By Definition 10.3, observe returns in bounded time regardless of observer state. The dispatch path is therefore decoupled from observer processing. □ □

11 Parallelism

11.1 Single-fragment dispatch

A single-fragment dispatch is largely sequential by Algorithm 1: the five subsystem hooks must execute in their stated order. Limited parallelism is available within E. eval if the executor supports it (e.g. parallel composition of sub-fragments), but the kernel's own pipeline is sequential.

11.2 Multi-fragment dispatch

When the kernel dispatches a stream of fragments, parallelism opportunities arise:

Proposition 11.1 (Per-Fragment Parallelism)**.** *If the fragments in a workload are pairwise independent (their executor evaluations do not share state), they may be dispatched in parallel on independent kernel instances or on a single kernel with thread-safe subsystems.*

Proof. By Theorem 6.2, the dispatch output depends only on the per-fragment subsystem interfaces. Independent fragments do not share subsystem state (provided the implementations are thread-safe). Hence parallel execution is semantics-preserving. □ □

Remark 11.2. *In practice, thread-safety of subsystems is a per-implementation property. The kernel architecture admits parallelism but does not enforce it; specific subsystem implementations may serialise on internal locks or be wait-free. The architecture is parametric in this choice.*

Part VII
Discussion

12 Comparison with Other Dispatch Substrates

12.1 Microkernels

The microkernel design (Liedtke, 1995; Klein et al., 2009) centres on a small set of primitives—thread, IPC, address space, capability—and delegates other functionality to user-space servers. Our dispatch kernel is similar in spirit: it provides a small set of primitives (the five subsystems) and delegates application-specific behaviour to subsystem implementations. The two designs differ in the granularity of dispatch: microkernels dispatch system calls, while our kernel dispatches symbolic fragments. The architectural decomposition is analogous.

12.2 Monolithic operating system kernels

Monolithic kernels (Tanenbaum and Bos, 2014) integrate validation, caching, scheduling, I/O, and monitoring tightly within the kernel address space. The interfaces between these subsystems are typically internal and not part of any stability contract. Our kernel exposes the five subsystems as interface-stable boundaries, making each independently replaceable. The trade-off is the per-call overhead of the interface abstraction (Corollary 10.2), which is $\mathcal{O}(1)$ but non-zero.

12.3 Virtual machines

The Java Virtual Machine (Lindholm et al., 2014) dispatches bytecode instructions through a fixed interpreter loop or JIT compiler. Our kernel is more general: it dispatches arbitrary fragments through a parametric executor. A JVM-like substrate can be built as an executor whose fragments are bytecode and whose eval runs the interpreter. The five-subsystem decomposition then applies: bytecode verification is PVE, the JIT compilation cache is CMM, thread scheduling is PSS, native I/O is DIC, and JVMTI events are TEM.

12.4 Actor systems

Actor systems (Hewitt et al., 1973; Agha, 1986; Armstrong, 2003) dispatch messages to receivers via mailboxes. The five-subsystem mapping is direct: message-format validation is PVE, mailbox memoization is CMM, mailbox scheduling is PSS, remote-actor lookup is DIC, and supervisor events are TEM. The actor abstraction differs from our kernel in its focus on concurrency and isolation; the dispatch substrate underneath is structurally similar.

12.5 Database query engines

Query engines (Graefe, 1993) dispatch query plans to physical operators. The mapping: plan validation is PVE, plan-cache or materialised-view lookup is CMM, query-plan scheduling is PSS, storage retrieval is DIC, and query-event logging

is TEM. The kernel's parametric design subsumes query-engine architecture.

12.6 Event-driven runtimes

Event-driven runtimes (Node.js, Erlang/OTP, Tokio) dispatch events to handlers via reactor loops (Schmidt et al., 2000). The five-subsystem mapping again applies. The reactor pattern is a refinement of the dispatch substrate in which the order of fragment arrival is non-deterministic and the scheduler (PSS) takes on a central role.

13 Limitations and Open Questions

13.1 Subsystem ordering

Algorithm 1 fixes the order of subsystem hooks: PVE before PSS before CMM before E before σ_{T}. Other orderings are possible (e.g. CMM before PVE, allowing cached results to bypass validation). The Independence Theorem (Theorem 6.2) is sensitive to the ordering: reordering can affect correctness if subsystems share state in ways the interface conceals. A principled treatment of alternative orderings and their preservation properties is outside our scope.

13.2 Subsystem composition

The five subsystems are presented as independent slots. In practice, applications may want to compose multiple validators (e.g. type checking + signature verification + capability check) into a single PVE. The composition is supported by ad hoc means (the composed PVE returns true iff every constituent returns true) but a general theory of subsystem-internal composition is left for future work.

13.3 Distributed dispatch

Our treatment assumes a single kernel instance. Distributed dispatch—multiple kernel instances cooperating to serve a single dispatch stream—raises questions of consistency between subsystem states (especially σ_{M}) and of failure handling that the present architecture does not address.

13.4 Subsystem evolution policies

The stability contract (Theorem 7.3) is conservative: additive evolution only, with semantic-versioning discipline. More flexible policies, such as soft-deprecation with backwards-compatibility shims, are common in production systems but require additional contract machinery.

14 Conclusion

We have specified a kernel architecture for the dispatch of categorical computation: a substrate that wraps an executor with five subsystems whose responsibilities partition the dispatch envelope. The five—pre-dispatch validation (PVE), post-dispatch memoization (CMM), pending-state scheduling (PSS), external-resource retrieval (DIC), and runtime invariant monitoring (TEM)—are derived from the universal dispatch responsibilities shared across operating-system kernels, virtual machines, actor systems, query engines, and event-driven runtimes.

The architecture rests on six structural results: the Dispatch Correctness Theorems (Theorems 5.2 and 5.3) which ensure the wrapped executor's semantics are preserved; the Independence Theorem (Theorem 6.2) which establishes pairwise functional independence of the five subsystems; the Stability Contract Theorem (Theorem 7.3) which guarantees client compatibility under additive kernel evolution; the Determinism Preservation Corollary (Corollary 5.4); the Compositional Substitution Theorem (Theorem 8.1); and the Conformance Decidability Theorem (Theorem 9.3) which provides a property-based regression framework.

The kernel is purely a substrate. It does not commit to a fragment grammar, a value space, a validation policy, a cache eviction strategy, a scheduler, a retrieval mechanism, or a monitor implementation. What it provides is the architectural skeleton over which such choices are made, together with the theorems that ensure the choices interact compositionally and that downstream clients depending on the frozen surface remain compatible across kernel evolutions.

The framework is foundational. It does not specify which executors should be wrapped, which subsystem implementations should be chosen, or which conformance predicates should be checked. Those choices are made by the applications that

consume the kernel. What the framework guarantees is that, once those choices are made, the resulting substrate satisfies the six structural theorems above: it dispatches correctly, the subsystems compose independently, clients of the frozen surface are stable, dispatch is deterministic given deterministic subsystems, bisimilar subsystems are interchangeable, and the conformance regression suite is decidable.

References

Gul Agha. *Actors: A Model of Concurrent Computation in Distributed Systems*. MIT Press, 1986.

Joe Armstrong. *Making Reliable Distributed Systems in the Presence of Software Errors*. PhD thesis, Royal Institute of Technology, Stockholm, 2003.

Andrew D. Birrell and Bruce Jay Nelson. Implementing remote procedure calls. *ACM Transactions on Computer Systems*, 2(1):39–59, 1984.

Goetz Graefe. Query evaluation techniques for large databases. *ACM Computing Surveys*, 25 (2):73–169, 1993.

Carl Hewitt, Peter Bishop, and Richard Steiger. A universal modular ACTOR formalism for artificial intelligence. In *Proceedings of the 3rd International Joint Conference on Artificial Intelligence (IJCAI)*, pages 235–245, 1973.

Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, et al. seL4: Formal verification of an OS kernel. In *Proceedings of the 22nd ACM SIGOPS Symposium on Operating Systems Principles (SOSP)*, pages 207–220, 2009.

Jörg Liedtke. On μ-kernel construction. *ACM SIGOPS Operating Systems Review*, 29(5):237–250, 1995.

Tim Lindholm, Frank Yellin, Gilad Bracha, and Alex Buckley. *The Java Virtual Machine Specification*. Addison-Wesley, java se 8 edition, 2014.

Douglas C. Schmidt, Michael Stal, Hans Rohnert, and Frank Buschmann. *Pattern-Oriented Software Architecture, Volume 2: Patterns for Concurrent and Networked Objects*. Wiley, 2000.

Andrew S. Tanenbaum and Herbert Bos. *Modern Operating Systems*. Pearson, 4th edition, 2014.